

glider-winds Documentation

This is an overview of the GitHub repository ‘glider-winds’, which comprises work completed as part of the University of Exeter undergraduate project ‘Winds from Gliders’. It aims to introduce the project and serves as a main reference point to allow others to understand the codebase and hopefully facilitate handover of the project.

Author: Toby Alexander, University of Exeter (toa201@exeter.ac.uk)

Contents

1. Introduction	1
2. Content and Structure	2
2.1. <i>pyopensky_client.ipynb</i>	2
2.2. <i>glider_analysis.ipynb</i>	2
2.3. <i>post_process.ipynb</i>	6
2.4. <i>ogn_tcp.ipynb</i>	7
3. Sample Data	8
4. Suggestions	9
A couple of points about the OGN data source.....	9
Recommended reading	9
5. Further Reference Material	10
Co-locating flights and radiosondes	10
Appendix	11

1. Introduction

With significant variation in wind observable over just a few hundred metres, detailed forecasting benefits from dense coverage of wind observations. Civil aircraft provide a wealth of observations at their cruising altitude of ~10km, but there are currently few observations in the lower atmosphere. Therefore, the aim of this undergraduate project has been to assess whether useful wind information over the UK can be obtained from gliders, which fly in this region of interest. A challenge arises because their broadcast data is limited to a basic GPS track, primarily for anticollision purposes, yet it is still possible (with some reasonable assumptions) to derive winds just from a glider’s motion through the atmosphere. Thermal soaring paths of gliders were modelled to extract horizontal drift of the aircraft due to wind, with estimates externally validated against radiosonde measurements and model reanalysis. A study of the business aspects investigated the current state of the UK receiver network and the feasibility of real time data acquisition for numerical weather prediction.

Thermals form when air heated by the Earth’s surface becomes buoyant and rises. They are characterised by cumulus clouds forming at their top, allowing glider pilots to spot them and climb with the rising air to extend flight time. Pilots circle within the thermal until the lift

diminishes at higher, cooler altitudes. The warm air experiences advection due to horizontal wind. Therefore, the circling glider is modelled as an approximately passive tracer in the wind field as it drifts horizontally to maintain its position within the core of the shifting thermal.

We are using data collected by [FLARM](#), which is a proprietary system created primarily for anticollision purposes. Gliders equipped with a FLARM system can broadcast their position and track to other local aircraft so that any potential collisions are flagged on a display in the cockpit. However, messages are broadcast passively, and so they can also be collected by ground receivers, allowing us to leverage an already existing network for our purposes.

2. Content and Structure

The repository centres around a series of Jupyter notebooks that were written to collect, process and analyse glider flight data. However, it also comprises other objects, including this documentation, a BSD-3-Clause license, sample data for testing and familiarisation, and other reference material that may be of use to future contributors.

The main Jupyter notebooks are as follows, with their function described in more detail further below:

1. ***pyopensky_client.ipynb***: obtain historical data of high-resolution flight tracks for our feasibility study. Achieved using ‘pyopensky’ module, which allows access to the OpenSkyNetwork’s Trino database of historical FLARM messages.
2. ***glider_analysis.ipynb***: examine individual flight tracks obtained from either *pyopensky.ipynb* or *ogn_tcp.ipynb*; identify thermal soaring events; model events via non-linear regression to extract wind estimates.
3. ***post_process.ipynb***: data cleaning; external validation via radiosonde and ERA5 comparison.
4. ***ogn_tcp.ipynb***: stream live FLARM messages from the Open Glider Network’s APRS client service.

The core work is contained within notebook 3, *glider_analysis.ipynb*. This handles the data processing and is therefore likely to be of most interest to future contributors.

2.1. *pyopensky_client.ipynb*

A SQL like query is constructed to request a subset of the historical FLARM data stored in the OpenSky Network’s Trino database. Queries were designed to pull glider flight messages that were close spatially and temporally to soundings, and specifically in regions of dense glider activity. Messages are then decoded and filtered, before being grouped into individual flight tracks for each of the gliders present in the queried region. Each flight is then saved out to file for further analysis.

2.2. *glider_analysis.ipynb*

Responsible for analysis of individual flight tracks. Conducts detection of thermal soaring events within a flight. Each event is then considered in turn for binning into smaller segments and model fitting. The recipe is as follows:

1. Load in data from a *.parquet* file into a pandas dataframe.

Parquet files were chosen for their storage efficiency, but csv, text file, etc could just as easily be used. You must set the root to the folder containing the flight file and set the filename. An out directory is where figures and results will be saved to. Set the source flag to indicate whether the flight file is from the OGN APRS (True) or the OpenSky Networks Historical Trino (False). The two sources have very slightly different data structures, so are handled differently.

2. Extract parameters of interest, project geospatial coords onto xy plane, and visualise flight track.

Main parameters are timestamps, latitude, longitude, and GPS altitude. We use a basic equirectangular projection method since we are working over distances of 10s of km, and so the error introduced is negligible. This is given by

$$\begin{aligned} x &= R_E(\phi - \phi_0) \cos(\psi_0) \\ y &= R_E(\psi - \psi_0) \end{aligned}$$

where R_E is the earth radius, ϕ is the longitude, ψ the latitude, ϕ_0 and ψ_0 reference longitude and latitude, taken as the midpoint of the flight path. A cosine term is included in the x projection because the lines of longitude get closer together at higher latitude.

3. Detect thermal soaring events within flight tracks via angular rate and vertical speed thresholding.

The algorithm creates an array of instantaneous angular rates ω (arr *db*). To achieve this, first the bearing between each pair of adjacent messages is found as $b_i = \arctan(\Delta y / \Delta x)$ where $\Delta x = x_{i+1} - x_i$ and $\Delta y = y_{i+1} - y_i$ give the 2D vectors between adjacent messages in the flight track. Then the gradient of b on t gives $\omega_i \sim b_i / \Delta t_i$ at each time interval. A similar process of vertical position (arr *alt_m*) on t gives z component of glider velocity (arr *dz*) via $\dot{z} = \Delta z / \Delta t$.

To identify thermal soaring events, a Boolean mask of the entire flight track is created by considering which elements of the arrays *db* and *dz* exceed the chosen threshold values of $\omega_{thr} > 0.02$ rad/s and $\dot{z}_{thr} > 0$ m/s. Then window segments in the flight where both thresholds are exceeded for at least 8 seconds are stored in a list of tuples containing the start and end indices. The 8 second threshold allows us to ignore segments that may be detecting noise.

Further processing merges identified thermal soaring windows that are separated by less than 20 seconds. We are assuming that these adjacent windows are from the same thermal soaring event. The gap may exist due to noise in the signal causing the detector values to drop below their thresholds briefly. Finally, these merged windows are screened for windows that contain more than 60s seconds of flight only. These windows are deemed long enough to be worth processing. This is because a typical turn through a full circle takes 30s, and we need at least 2 turns to process the soaring event.

4. Inspect detected thermal soaring events

Thermal soaring windows can be trimmed down manually before fitting to remove any transient tails where the glider is entering/exiting the circling motion. These can sometimes be present

despite the thresholding algorithm, but they need to be removed as they can cause issues with the fitting procedure.

5. Bin the chosen thermal soaring window into segments based on turns/time periods.

Depending on atmospheric conditions, an exceptional thermal soaring event can span well over 1km of vertical distance, containing many turning circles within it. So, to improve vertical resolution of the measurement, thermal soaring events are split up into smaller segments, aiming for each bin to have two full turns/time periods within each. This also aids the model fitting: the model (currently! 😊) assumes a constant turn rate ω across the segment of data it is fed, but we have observed that ω often decreases over the course of a soaring event and can change by as much as 20%. This may be due to the area of lift spreading out over a greater horizontal area at higher altitudes, and so the pilot chooses to relax to a shallower bank angle to improve their aerodynamics, which causes them to arc out wider, slower circles. Anyway, the bottom line is that feeding in the whole soaring window means the model would not converge to a solution. However, over just two turns, ω hardly changes, so the model can handle this.

Binning is achieved by first getting a rough estimate for the time period that it takes a glider to go through a full turn during this specific soaring window. First, we look our window and split into our x and y components as functions of time. In this example, we can see the glider is drifting almost entirely eastward, so most of the wind will be in this direction.

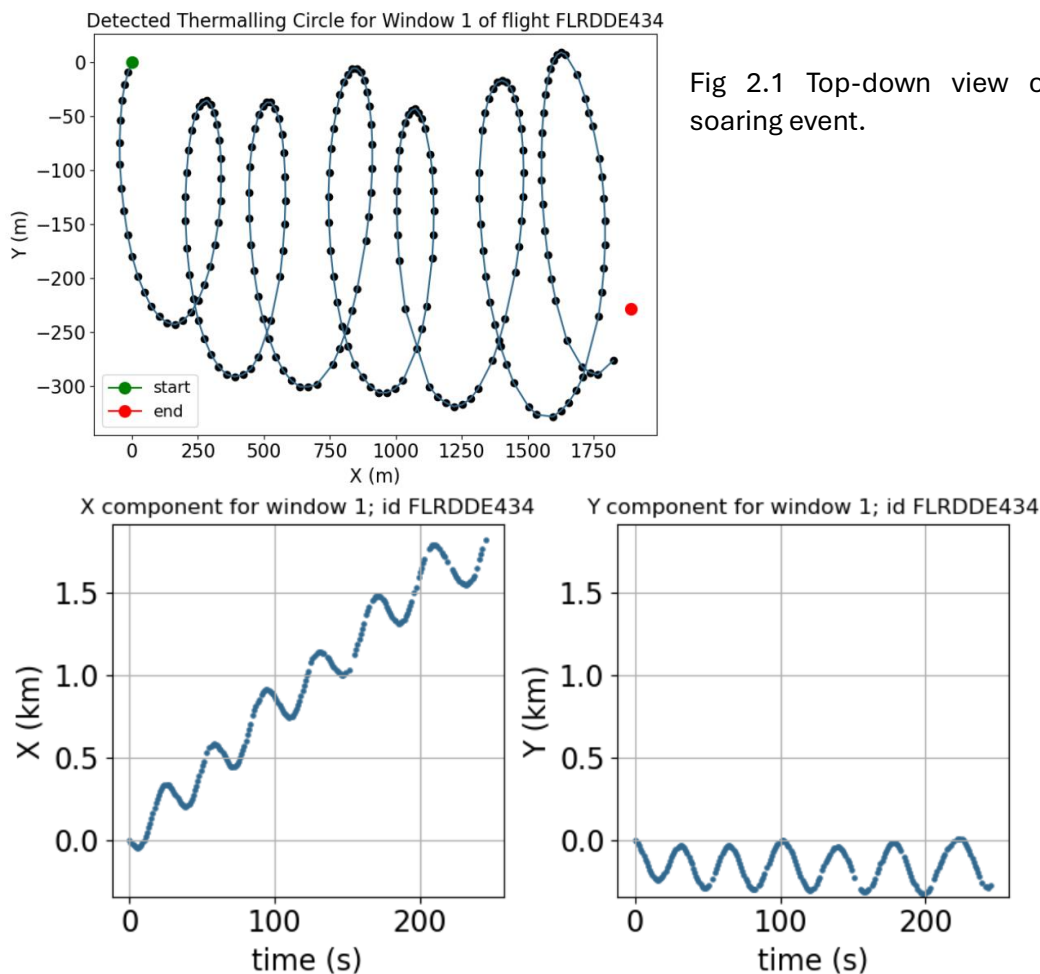


Fig 2.2. x and y components of motion. We see a sinusoid combined with a linear drift, mostly in the x (eastward) direction.

We want to isolate the sinusoidal component, so we do a simple linear regression and take the residuals, which should just contain the oscillations, as below. Then we follow a similar process to find an angular rate as we used for the thresholding algorithm. Except this time we have an approximately constant ω so we use a linear regression of bearing on time rather than finding the instantaneous values.

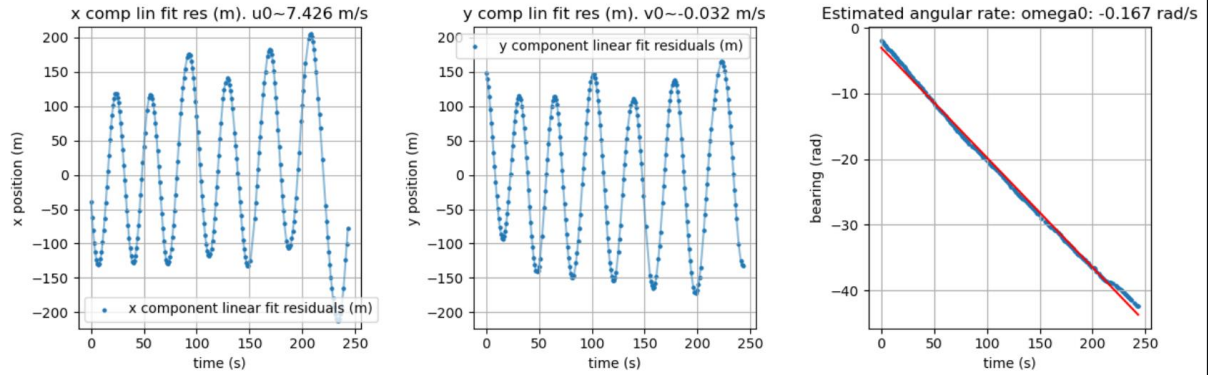


Fig 2.3. **left, middle:** x and y residuals of a linear regression of a thermal soaring event. **right:** linear regression (red) of bearing on time data (blue) to extract an angular rate.

Now we have a global angular rate for the entire thermal soaring window, we can relate it to the oscillation's time period P via

$$P = 2\pi/\omega$$

Then the soaring event is split into bins, each spanning a time $2P$.

6. Fit each bin to the thermal soaring model.

Horizontal motion is decomposed into time dependent x and y components as follows:

$$x(t) = R\cos(\omega t + \phi) + ut + x_0 \quad (1)$$

$$y(t) = R\sin(\omega t + \phi) + vt + y_0 \quad (2)$$

with R the radius of circular motion; ω the turn rate of the aircraft; ϕ the phase angle at which the glider enters the circular motion; (u, v) the horizontal 2D wind drift vector, and (x_0, y_0) offsets.

Each bin within a soaring window is iterated over and fitted to the model with its own set of parameters via scipy's least_squares algorithm. The data is non-dimensionalised during the fitting process as this was observed to greatly improve numerical stability. Parameters are re-dimensionalised after the procedure.

Parameter Initialisation

We can make some initial guesses at what our fitting parameters in our model should be. These can be fed into the fitting procedure to help the process to converge more quickly to a correct solution. We need $[R_0, \omega_0, \phi_0, u_0, v_0, x_{00}, y_{00}]$.

ω_0 estimate

Identical method to global estimate of angular rate used for binning. Conduct initial linear fit of each component of position to find approximate linear drift (can be fed in as (u_0, v_0)). Subtract from the data to remove the drift term and leave behind only the oscillatory term. Find bearings between adjacent (x, y) points in time, then find a turn rate with a linear fit of bearing against time.

 R_0 estimate

Use the fact that we can relate the amplitude of a sinusoid of the form $f = R\cos(\omega t)$ to the root mean square value of the function:

$$R_0 \sim \sqrt{2} f_{rms}$$

This is only true for an integer number of time periods. Fortunately, segments of thermal soaring are binned so that each contains approximately 2 periods.

 ϕ_0 estimate

$$\text{Suppose } f = R\cos(\omega(t - c)) = R\cos(\omega t + \phi)$$

We know that, when $t = c$ (i.e. with an argument of zero), a cosine function returns 1, so

$$f(t = c) = R$$

So, then find the value in the signal that is closest to our R_0 estimate from above, which will occur at the corresponding value in time, c .

And then from our definition of the function f , we know that with $t = c$

$$\omega(t - c) = \omega t + \phi$$

$$\phi \sim -\omega c$$

This is going to carry through both the error from our omega estimate and R estimate but generally can get us reasonably close.

Post fitting

The covariance matrix is estimated from the Jacobian matrix and variance on each parameter extracted from the diagonal elements. A data frame is constructed containing the fitted parameters for each bin along with associated uncertainties. Other columns are included such as a reference timestamp and location of the measurement for use in validation. This dataframe can be saved out to a csv file for feeding into *post_process.ipynb*. Data is also visualised to show the data for each bin, with its corresponding model overlaid, and model residuals included for examination.

2.3. *post_process.ipynb*

This notebook comprises some of the data cleaning and validation, but is less well documented than I would like, so may be of less use than others. I include it because there are a few sections that may be helpful, especially: Section 4, which deals with validation against sonde data; and

Section 5, which handles obtaining ERA5 reanalysis data for validation. Here I briefly describe each section:

1. Load in data

Reads in csv file we saved out from *glider_analysis.ipynb*. If you have processed multiple flights from a day, you can combine all the results from each flight into one csv and read that in. Also reads in radiosonde data, as a csv file produced by the University of Wyoming radiosonde archive (<https://weather.arcc.uwyo.edu/upperair/sounding.shtml>)

2. Estimate upper bound on uncertainty in wind measurements

Currently, our uncertainty in wind measurement is only characterised by the formal fit error. Additional uncertainties (model inadequacy, GPS noise, variability in ω and R) are addressed by comparing adjacent bins, assuming inter-bin variation reflects measurement error. This may overestimate error due to potential wind shear, so provides an upper bound. The final uncertainty is taken as the larger of the fit error and inter-bin variation.

Quality control thresholds of 1 m/s and 2.5 m/s are applied to the formal fit error (lower bound) and adjacent bin uncertainty (upper bound) respectively. Discarded bins are associated with poor sinusoidal fits characterised by large residuals and large inter-bin differences. The thresholds were validated against radiosonde wind shear data, which showed that real shear contributions at 500-2000m altitude for the days we analysed are typically below 1.5 m/s per 100m. With bin sizes less than 100m, any inter-bin variation exceeding the wind shear can be regarded as unphysical, confirming that discarded bins reflect poor model fits rather than genuine wind variation.

3. Clustering of measurements

Measurements are clustered together into groups that are separated by no more than 50 km and ~2 hours. The group that is closest in time and space to the radiosonde is used for validation.

4. Quantitative comparison between glider measurements and radiosonde

Each glider measurement in the chosen cluster is matched with the sonde data point with the closest match in altitude. The disagreement between the two measurements is calculated. Plots visualise the distribution of residuals.

5. Quantitative comparison between glider measurement and ERA5

This section comprises a method of retrieving ERA5 data from the Copernicus Climate Data Store. The example queries can select wind components at selected pressure levels, over a given time period and spatial extent, specified as a geospatial bounding box. The code then cross references the ERA5 pressure levels with the sonde data (which includes pressure and GPS altitude) to convert them to altitudes. And it then matches each glider measurement with its closest ERA5 grid point based on the reference position and timestamps stored in the fitting results dataframe.

2.4. *ogn_tcp.ipynb*

This notebook sources data from the Open Glider Network, which is a non-profit, community led project. It comprises four sections. First, a TCP connection to the APRS server allows FLARM

messages to be streamed live for a chosen time period and then stored in a list of strings. The second stage parses the raw messages into a *pandas* data frame for easy manipulation, before the data is filtered for messages originating only from gliders, cleaned of any erroneous messages and visualised. Finally, messages are grouped into unique flight tracks and examined before saving out to individual files for each flight. This would then allow for further processing and analysis to extract wind estimates.

3. Sample Data

Two sample datasets from the OGN APRS are provided, described below. Each sample has:

- Parsed and filtered *pandas* dataframe containing only UK flying glider messages, stored as a *.parquet* file.
- Several *pandas* dataframes containing extracted individual flight tracks, stored as *.parquet* files. See appendix for flight track plots.

Name	OGN_sample_1	OGN_sample_2
Date	22/03/2026	22/03/2026
Timespan	11:47-12:17	15:34-16:04
Total glider messages within UK bounding box	30409	62693

Table 1. Description of OGN sample datasets.

Here I have included the data fields in the parsed OGN messages. The OGN wiki has additional information on the message structure, as well as units for some of the parameters:

<http://wiki.glidernet.org/wiki:ogn-flavoured-aprs>

#	Column	Dtype	#	Column	Dtype
0	raw_message	object	31	real_address	object
1	reference_timestamp	datetime64[ns,UTC]	32	reserved	float64
2	aprs_type	object	33	version	object
3	beacon_type	object	34	platform	object
4	name	object	35	cpu_load	float64
5	dstcall	object	36	free_ram	float64
6	receiver_name	object	37	total_ram	float64
7	latitude	float64	38	ntp_error	float64
8	longitude	float64	39	rt_crystal_correction	float64
9	symboltable	object	40	cpu_temp	float64
10	symbolcode	object	41	senders_visible	float64
11	timestamp	datetime64[ns,UTC]	42	latency	object
12	track	float64	43	senders_total	float64
13	ground_speed	float64	44	rec_crystal_correction	float64
14	altitude	float64	45	rec_crystal_correction_fine	float64
15	address_type	float64	46	rec_input_noise	float64
16	aircraft_type	float64	47	senders_signal	float64
17	no-tracking	object	48	senders_messages	float64
18	stealth	object	49	good_senders_signal	float64

19	address	object	50	good_senders	float64
20	climb_rate	float64	51	good_and_bad_senders	float64
21	flightlevel	object	52	relay	object
22	user_comment	object	53	voltage	float64
23	turn_rate	float64	54	wind_direction	float64
24	signal_quality	float64	55	wind_speed	float64
25	frequency_offset	float64	56	wind_speed_peak	float64
26	gps_quality	object	57	temperature	float64
27	error_count	float64	58	rainfall_1h	float64
28	signal_power	object	59	rainfall_24h	float64
29	software_version	float64	60	humidity	float64
30	hardware_version	float64	61	barometric_pressure	float64

Table 2. Parameters contained within OGN FLARM messages

4. Suggestions

A couple of points about the OGN data source.

The OGN data seems to be richer in the parameters it holds, when compared with the historical data we have been using. Most notable are #12 and #23, which are ‘track’ and ‘turn rate’. I am wondering if they would give a more accurate, less noisy idea of ω compared with the instantaneous values we have been calculating from the GPS coordinates. It may be worth looking into the FLARM protocol to determine where these parameters come from. As far as I know, FLARM devices do not have inertial measurement units inside, but I could be wrong. In the worst case, the FLARM device’s turn rate calculation is also based on GPS position, but at least if the calculation is happening locally in the device, it will be based on all of the messages available. Therefore, it might still be more accurate than our calculation, since the messages we have are limited to the ones that made it to the ground receiving station.

Improving the turn rate measurement would surely help the thermal soaring detection algorithm and potentially the binning process.

One small unresolved issue: OGN APRS data seems to include messages with matching timestamps. This causes divide by zero issues when taking time derivatives. For now, I have altered the code to discard matching timestamp messages and only keep the first message with that timestamp. Perhaps it is worth investigating whether these messages are duplicates or not. We found with the historical data from OpenSky Network that the message decoder outputs a truncated timestamp with a resolution of seconds, so perhaps this is also the case with the OGN message parser. We were able to retrieve a nanosecond resolution timestamp from the historical database, so we could get around this truncation. Solving this for OGN APRS could allow for higher resolution data.

Recommended reading

[1] Olive X, Strohmeier M, Sun J, Tresoldi G. *OpenSky Report 2023: Low Altitude Traffic Awareness for Light Aircraft with FLARM. 2023 IEEE/AIAA 42nd Digital Avionics Systems Conference; 2023 Oct 1-5; Barcelona, Spain.*

This is written by some of the people behind the OpenSky Network. They were our historical source of data, and they also have a live API, similar to OGN, but their receiver network in the UK is almost non-existent. Anyway, this article gives a nice introduction to FLARM, and they also outline their own thermal soaring detection algorithm!

[2] *A Network of Mode-S Receivers for Routine Acquisition of Aircraft-Derived Meteorological Data*. Edmund Keith Stone and Gary Pearce.

https://journals.ametsoc.org/view/journals/atot/33/4/jtech-d-15-0184_1.xml

Mode-S Winds from aircraft, which you may already be familiar with. A similar project on a larger scale. The bottom line is they were able to use aircraft pressure sensors to measure airspeed, which makes the wind measurement straightforward. We don't have those on gliders, so our job is trickier.

[3] *Glider Handbook Chapter 10: Soaring Techniques*. Federal Aviation Administration.

https://www.faa.gov/regulations_policies/handbooks_manuals/aviation/glider_handbook/gfh_chapter_10.pdf

Mainly for interest and for an insight into glider behaviour.

5. Further Reference Material

Co-locating flights and radiosondes

Previously there have been discussions around requesting a well-timed and placed radiosonde launch for validation purposes. In the hopes of informing this launch, here is a list of some of the operational sounding stations I am aware of in the UK, along with their proximity to nearby gliding clubs.

Place	Notes
○ Camborne (50.22N, 5.32W)	Radiosonde station — no dedicated gliding club within practical distance. Dartmoor GS is ~60km away.
○ Herstmonceux (50.89N, 0.32E)	Radiosonde station — East Sussex GC only ~8km away. Excellent pairing.
○ Watnall (53.00N, 1.25W)	Radiosonde station — Darlton GC ~35km, D&L GC ~55km.
○ Albemarle (55.02N, 1.88W)	Radiosonde station — Northumbria GC ~20km, Borders GC ~60km.
○ Aberporth (52.13N, 4.57W)	Radiosonde station — Black Mountains GC ~80km. Limited nearby gliding.

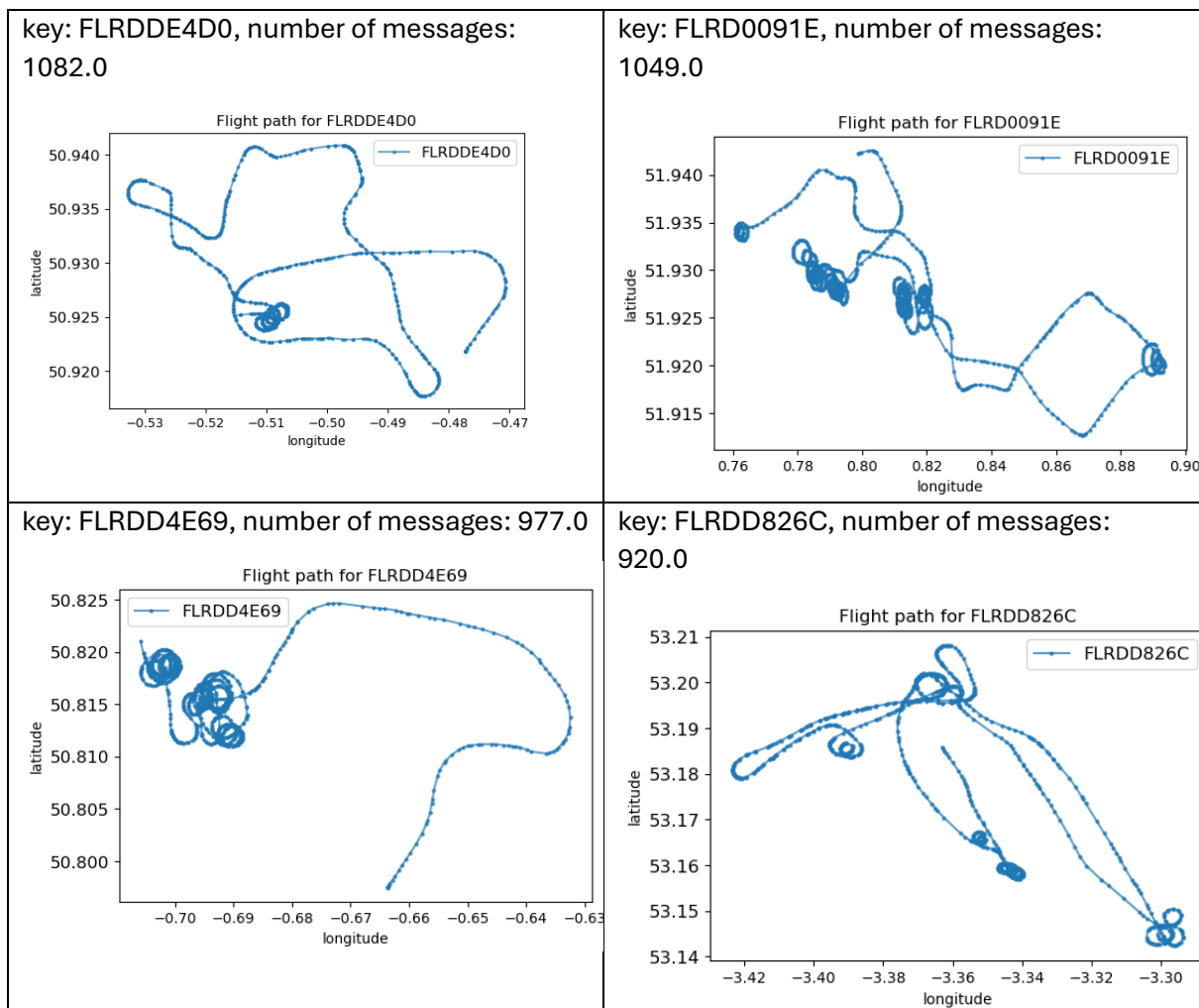
○ Hemsby (52.68N, 1.68E)	Radiosonde station — Norfolk GC ~35km, Rattlesden ~60km.
○ Larkhill (51.20N, 1.80W)	Radiosonde range station — Army GC Wyvern ~15km, Bath Wilts & N Dorset GC ~30km, Shalbourne GC ~25km. Rich gliding cluster.

Table 3. Locations of known sounding release stations in the UK and corresponding nearby gliding clubs.

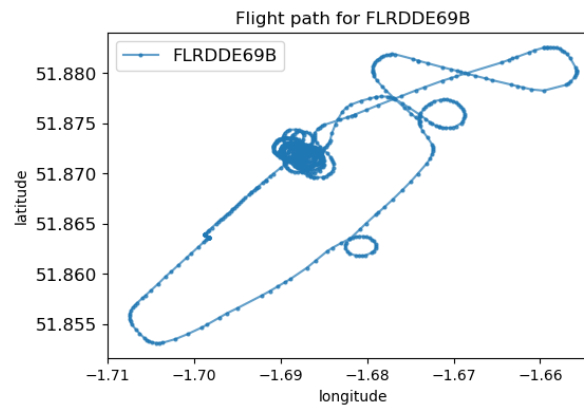
Appendix

Flight tracks for some of the most active flight IDs extracted from the OGN sample datasets.

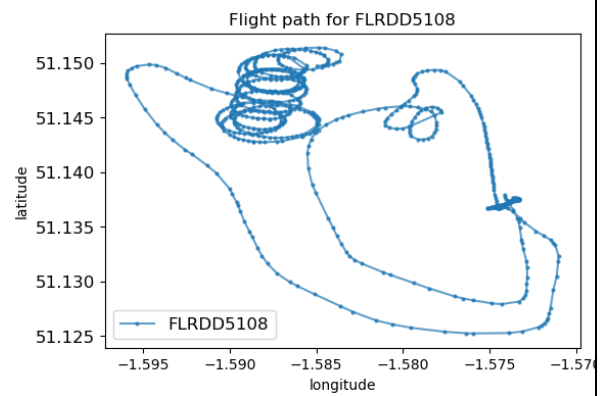
OGN_sample_1



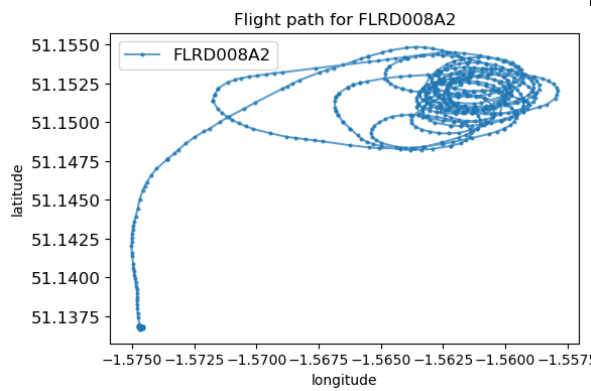
key: FLRDDE69B, number of messages: 834.0



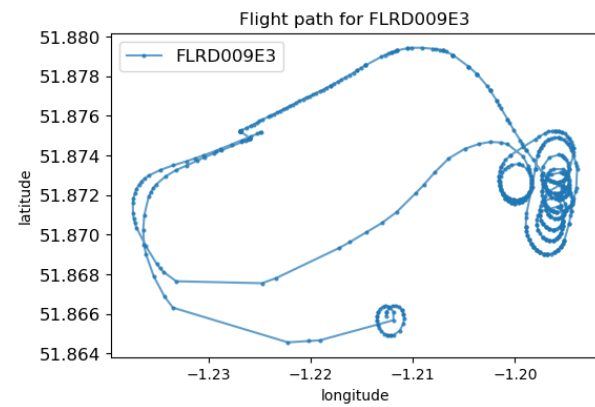
key: FLRDD5108, number of messages: 790.0



key: FLRD008A2, number of messages: 773.0

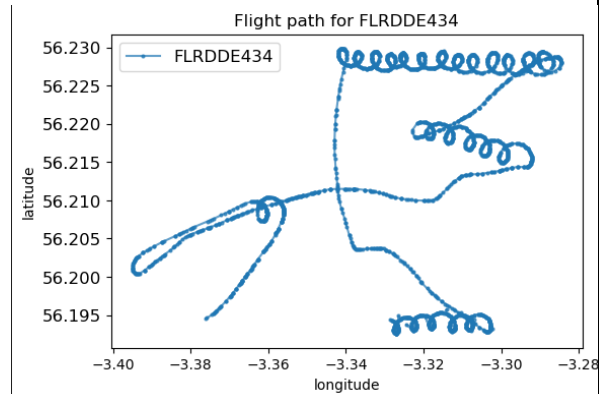


key: FLRD009E3, number of messages: 745.0

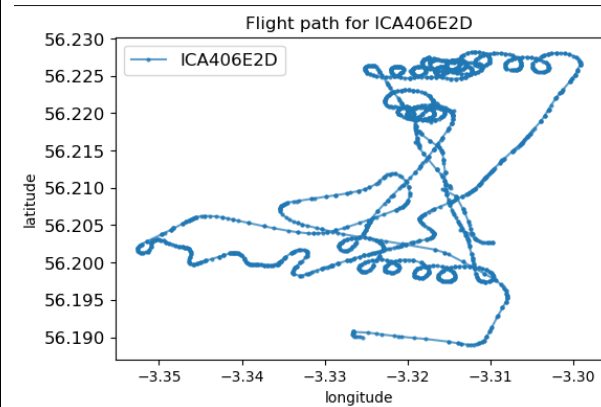


OGN_sample_2

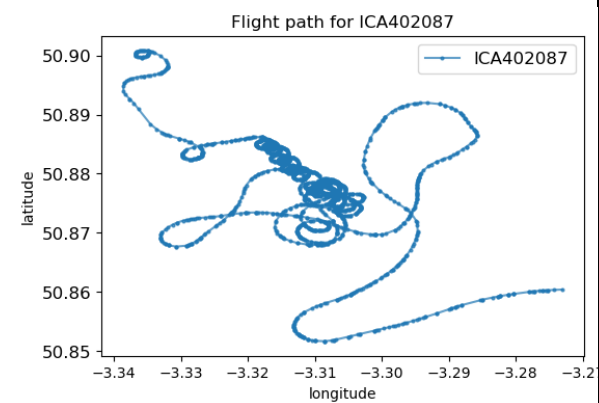
key: FLRDDE434, number of messages: 2616



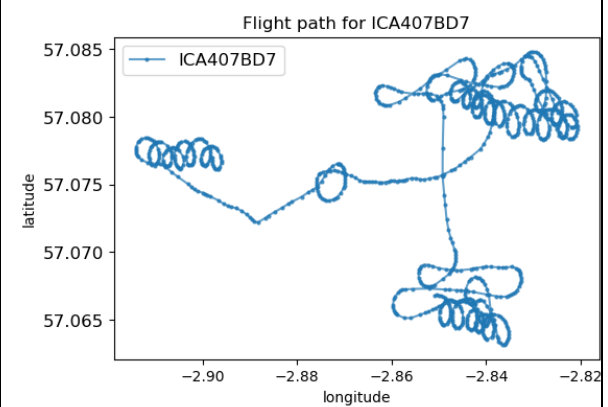
key: ICA406E2D, number of messages: 2371



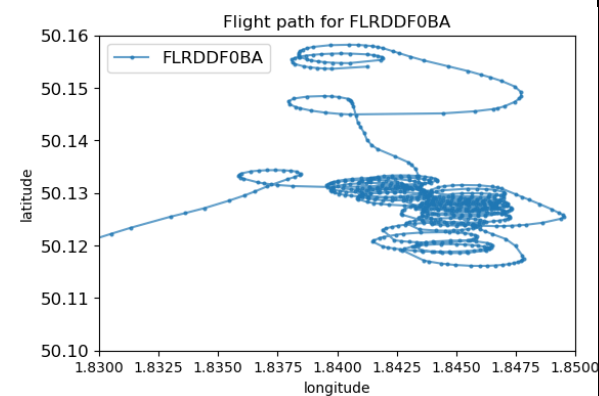
key: ICA402087, number of messages: 2068



key: ICA407BD7, number of messages: 1050



key: FLRDDF0BA, number of messages: 914



key: FLR57F46C, number of messages: 801

